AptiSense Viva and Presentation Guide

Project Title
AptiSense: AI-Powered Aptitude Assessment and Placement Readiness Prediction System

1. One-Line Introduction
AptiSense is a web-based aptitude assessment platform that helps students take mock tests, analyze their performance, and predict their placement readiness using analytics and an ML service.

2. Main Goal of the Project
The goal of the project is to move beyond simple test scores and provide meaningful insights like readiness score, success probability, weak-topic identification, and batch-level analytics.

3. Tech Stack
Frontend: React, TypeScript, Vite
Backend: Node.js, Express.js
Database: MongoDB
ML Service: Python, FastAPI
Charts: Recharts
Authentication: JWT-based login

4. Main Parts of the Project
Frontend: Displays student and admin dashboards and handles user interaction.
Backend: Handles authentication, tests, scoring, analytics, and DB communication.
ML Service: Calculates readiness score, success probability, and weak topics.
Database: Stores users, tests, results, and predictions.

5. Working Flow
1. User logs in.
2. Student selects and starts a mock test.
3. Student answers aptitude questions section by section.
4. Backend evaluates answers after submission.
5. Backend computes section scores, average score, accuracy, and time taken.
6. Backend stores the test result in MongoDB.
7. Backend sends performance features to the ML service.
8. ML service returns readiness score, success probability, weak topics, and feature contributions.
9. Student dashboard displays analytics and prediction results.
10. Admin dashboard displays batch-level insights and risk categories.

6. Student Dashboard
Shows placement success probability, readiness score, weakness alert, tests taken count, skill radar chart, performance trend, and available mock tests.

7. Admin Dashboard
Shows total students, safe/moderate/at-risk categories, batch heatmap, weak-topic analysis, and student performance table.

AptiSense Viva and Presentation Guide

8. Important Database Collections
User: stores name, email, password, role.
Test: stores title, pattern, time limit, sections, questions, options, correct answers.
TestResult: stores section scores, average score, accuracy, time taken, and references.
Prediction: stores success probability, readiness score, weak topics, and shap-like values.

9. How Scoring Works
The backend compares selected options with the correct answers and calculates section-wise and overall performance metrics. These results are saved in the database.

10. How Prediction Works
The ML service uses performance features such as quant score, logical score, verbal score, average score, test count, and attempt quality. Based on these inputs, it calculates a readiness score and success probability and identifies weak topics.

11. Key Viva Answer
If asked whether this is a full trained ML model, answer honestly:
The current version uses weighted predictive logic through a separate ML service, and it can be upgraded later to a fully trained machine learning model using real student placement data.

12. Advantages
1. Easy-to-use interface
2. Automated test evaluation
3. Section-wise analytics
4. Readiness prediction
5. Weak-topic identification
6. Useful for both students and admins

13. Limitations
1. Prediction logic is currently basic
2. Focused only on aptitude sections
3. Depends on proper backend, database, and ML setup
4. Does not yet support coding or interview rounds

14. Future Scope
1. Add coding assessment
2. Add personalized recommendations
3. Use trained ML models on real datasets
4. Support company-specific test patterns
5. Deploy as a cloud-based platform

15. 60-Second Presentation Script
My project is AptiSense, an AI-powered aptitude assessment and placement readiness prediction system. It helps students prepare for placements by providing mock aptitude tests, performance analytics, and predictive insights. The frontend is built with React and TypeScript, the backend uses Node.js

and Express, MongoDB stores the data, and a Python FastAPI service generates prediction insights. After a student submits a test, the backend calculates scores, stores results, sends performance features to the ML service, and then shows readiness score, success probability, and weak topics on the student dashboard. Admins can also monitor batch performance and topic-wise weaknesses.

16. Common Viva Questions and Short Answers

Q1. What is AptiSense?
A web-based aptitude assessment and placement readiness prediction platform.

Q2. Why did you build this project?
To help students understand readiness and weak areas, not just raw marks.

Q3. Why use React?
To build a reusable and interactive frontend with dashboards.

Q4. Why use MongoDB?
Because the project data fits well into document-based storage.

Q5. Why use FastAPI?
Because it is lightweight and efficient for ML-related APIs.

Q6. What does the backend do?
It handles authentication, tests, scoring, analytics, and DB operations.

Q7. What does the ML service do?
It predicts readiness score, success probability, and weak topics from performance features.

Q8. What is JWT used for?
JWT is used for secure login and protected API access.

Q9. What is the role of the admin?
To monitor batch performance and identify students at risk.

Q10. What is future scope?
Coding assessment, stronger ML models, and personalized recommendations.

17. Final Survival Template for Viva

If you forget details, say:
The frontend handles the user interface.
The backend handles business logic and database operations.
The ML service predicts readiness from test performance features.
MongoDB stores users, tests, results, and predictions.